

Exercise 1

From Infix to Postfix

Exercise 2

Longest Valid Parentheses Substring

Given a string **str** consisting of opening and closing parenthesis '(' and ')', the task is to find the length of the longest valid parentheses substring.

Examples:

Input: *str = "(()"*

Output: *2*

Explanation: *Longest Valid Parentheses Substring is "()".*

Input: *str = "()()())"*

Output: *4*

Explanation: *Longest Valid Parentheses Substring is "()()".*

Solution:

// C++ program to find length of the
// longest valid substring

```
#include <iostream>
```

```
#include <stack>
```

```
using namespace std;
```

```
int maxLength(string s) {
```

```
    stack<int> st;
```

```
    // Push -1 as the initial index to
```

```
    // handle the edge case
```

```
    st.push(-1);
```

```
    int maxLen = 0;
```

```
    // Traverse the string
```

```
    for (int i = 0; i < s.length(); i++) {
```

```
        // If we encounter an opening parenthesis,
```

```
        // push its index
```

```
        if (s[i] == '(') {
```

```

        st.push(i);
    } else {
        // If we encounter a closing parenthesis,
        // pop the stack
        st.pop();

        // If stack is empty, push the current index
        // as a base for the next valid substring
        if (st.empty()) {
            st.push(i);
        } else {
            // Update maxLength with the current length
            // of the valid parentheses substring
            maxLength = max(maxLen, i - st.top());
        }
    }
}

return maxLen;
}

int main() {
    string s = ")()()";
    cout << maxLength(s) << endl;

    return 0;
}

```

Exercise 3

Form minimum number from given sequence

Auxiliary Given a pattern containing only I's and D's. I for increasing and D for decreasing. Devise an algorithm to print the minimum number following that pattern. Digits from 1-9 and digits can't repeat.

Examples:

Input: D	Output: 21
Input: I	Output: 12
Input: DD	Output: 321
Input: II	Output: 123
Input: DIDI	Output: 21435
Input: IIDDD	Output: 126543
Input: DDIDDIID	Output: 321654798

Solution:

```
// C++ program to print minimum number that can be formed
```

```
// from a given sequence of Is and Ds
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Function to decode the given sequence to construct
```

```
// minimum number without repeated digits
```

```
void PrintMinNumberForPattern(string seq)
```

```
{
```

```
    // result store output string
```

```
    string result;
```

```
    // create an empty stack of integers
```

```
    stack<int> stk;
```

```
    // run n+1 times where n is length of input sequence
```

```
    for (int i = 0; i <= seq.length(); i++)
```

```
    {
```

```
        // push number i+1 into the stack
```

```
        stk.push(i + 1);
```

```
        // if all characters of the input sequence are
```

```
        // processed or current character is 'I'
```

```
        // (increasing)
```

```

        if (i == seq.length() || seq[i] == 'I')
        {
            // run till stack is empty
            while (!stk.empty())
            {
                // remove top element from the stack and
                // add it to solution
                result += to_string(stk.top());
                result += " ";
                stk.pop();
            }
        }
    }

    cout << result << endl;
}

// main function
int main()
{
    PrintMinNumberForPattern("IDID");
    PrintMinNumberForPattern("I");
    PrintMinNumberForPattern("DD");
    PrintMinNumberForPattern("II");
    PrintMinNumberForPattern("DIDI");
    PrintMinNumberForPattern("IIDDD");
    PrintMinNumberForPattern("DDIDDIID");
    return 0;
}

```

Exercise 4

Longest Valid Parentheses Substring

Problem Statement:

Given a string s, return one of the longest palindromic substring in s.

Example 1:

Input: "babad"

Output: "bab" or "aba"

- Both "bab" and "aba" are valid longest palindromic substrings in the string "babad".

- **Example 2:**
Input: "cbbd"
Output: "bb"
The longest palindromic substring in "cbbd" is "bb".

Exercise 5

Shortest Subarray to be Removed to Make Array Sorted

Problem Statement:

Given an integer array arr, remove a subarray (can be empty) from arr such that the remaining elements in arr are non-decreasing. Return the length of the shortest subarray to remove. A subarray is a contiguous subsequence of the array.

Example 1:

Input: arr = [1,2,3,10,4,2,3,5]

Output: 3

Explanation: The shortest subarray we can remove is [10,4,2] of length 3. The remaining elements after that will be [1,2,3,3,5] which are sorted. Another correct solution is to remove the subarray [3,10,4].

Example 2:

Input: arr = [5,4,3,2,1]

Output: 4

Explanation: Since the array is strictly decreasing, we can only keep a single element. Therefore we need to remove a subarray of length 4, either [5,4,3,2] or [4,3,2,1].

Solution:

```
class Solution {
public:
    int findLengthOfShortestSubarray(vector<int>& arr) {
        int n = arr.size();
        int left = 0, right = n - 1;

        // 左边最长非递减前缀
        while (left + 1 < n && arr[left] <= arr[left + 1]) ++left;

        if (left == n - 1) return 0; // 已经是非递减的

        // 右边最长非递减后缀
        while (right > 0 && arr[right - 1] <= arr[right]) --right;

        int ans = min(n - left - 1, right); // 删除左边或右边

        int i = 0, j = right;
        while (i <= left && j < n) {
            if (arr[i] <= arr[j]) {
```

```

        ans = min(ans, j - i - 1);
        ++i;
    } else {
        ++j;
    }
}

return ans;
}
};

```

Exercise 6

Addition and Extraction of the array-based Maxheap.